American University of Beirut
Department of Electrical and Computer Engineering
EECE 350 - Computing Networks

Project Report

Πthon Arena: Snake Royale

Team Members and Workload Distribution

Team Member	ID	Email	Main Responsibility	Workload
Maher Karam	202507645	mrk35@mail.aub.edu	Server core and protocol	33.33%
Marc-Anthony Bou Mikhael	202501267	mab11@mail.aub.edu	Game engine and state rules	33.33%
Alexandre Yammine	202504320	awy05@mail.aub.edu	Client GUI, lobby, customization, and result screens	33.33%

1. Executive Summary

Snake Royale implements Πthon Arena as a centralized client-server multiplayer game written in Python using Pygame for the graphical interface. Two players connect to a server, register unique usernames, customize their snakes, challenge each other, and play a real-time battle in which the server owns all game logic. The implementation also includes the required advanced features: peer-to-peer text chat, spectator mode with fan cheering, and a creative crown power triggered by a purple pie.

The strongest design choice in the project is that clients are not trusted to compute the match. They only send input events and render server updates. This keeps both screens synchronized, prevents clients from inventing scores or collisions, and makes spectators possible because the same state stream can be sent to players and viewers.

2. System Architecture

The system follows a centralized client-server architecture. The server is the authority for registration, lobby status, matchmaking, match timing, snake movement, pie generation, obstacle collisions, score/health updates, spectators, cheering, and game-over decisions. Each client is responsible for rendering screens, collecting keyboard/mouse input, sending commands, and displaying received state.

Layer / File Group	Main Responsibility	Design Reason
server.py	Accepts TCP clients, validates usernames, manages lobby state, accepts/rejects challenges, runs the game loop, broadcasts updates, supports spectators and chat routing.	Central authority avoids inconsistent game states and keeps all players synchronized.
protocol.py	Defines the JSON message format, message types, length-prefix framing, and send/receive helpers.	A structured protocol is easier to debug and prevents TCP stream-splitting bugs.
game_engine.py and models.py	Holds board state, snake state, pie/obstacle models, collision checks, health rules, timer, crown power, and result logic.	Separating game rules from networking makes testing and future changes easier.
client.py and network.py	Creates the Pygame app, opens sockets, listens for server messages on a background thread, sends player actions and UI requests.	Keeps the interface responsive while network messages arrive asynchronously.
Screen modules	Connection, username, lobby, customization, gameplay, chat, spectating, and result screens.	Screen separation keeps the GUI maintainable and avoids one large monolithic client file.

Concurrency is handled on the server using one client-handling thread per connected socket and one separate match-loop thread for the active game. Shared dictionaries for clients, usernames, statuses, customizations, and the current match are protected by a lock. TCP_NODELAY is enabled to reduce input latency, and SO_KEEPALIVE is used to improve connection robustness.

3. Communication Protocol

The application protocol runs over TCP. Each message is encoded as UTF-8 JSON and preceded by a 4-byte network-order unsigned integer representing the JSON payload length. This solves the main TCP problem: a single recv() call may return half a message or multiple messages. The receiver first reads exactly 4 bytes, extracts the length, then reads exactly that number of bytes before decoding JSON.

Message Group	Types	Sender -> Receiver	Purpose
Registration	REGISTER, REGISTER_OK, REGISTER_FAIL	Client -> Server / Server -> Client	Connect a username and reject empty or duplicate names.
Lobby	USER_LIST, CUSTOMIZATION_UPDATE	Server -> Clients / Client -> Server	Publish online players, status, and selected snake color.
Matchmaking	CHALLENGE, CHALLENGE_RECEIVED, CHALLENGE_ACCEPT, CHALLENGE_REJECT	Client <-> Server	Start a match only when both players agree and are idle.
Gameplay	GAME_START, INPUT, GAME_STATE, GAME_OVER	Both directions	Send player directions and broadcast authoritative board state and results.
Chat	CHAT_SEND, CHAT_MESSAGE	Client -> Server -> Target Client	Route direct text messages between online users.
Spectators / Fans	SPECTATE_REQUEST, LEAVE_SPECTATE, LEAVE_SPECTATE_OK, CHEER	Client -> Server / Server -> Clients	Allow viewers to watch a running match and send timed cheering messages.
Error handling	INFO, ERROR	Server -> Client	Report accepted actions, rejected actions, or invalid requests.

The protocol is intentionally command-based rather than sending raw objects or Python pickles. JSON is readable in logs, language-independent, and safe enough for this scope. Length-prefix framing is more reliable than newline framing because chat messages and future payloads can contain arbitrary text. The tradeoff is slightly larger messages, which is acceptable because the board state is small.

4. Feature Implementation Status

Feature	Status	Evidence in Implementation	Notes
Server launched with port argument	Successfully implemented	server.py reads python server.py <port> and binds on 0.0.0.0.	Also prints 127.0.0.1 and LAN IP guidance.
Client connects using IP and port	Successfully implemented	Connect screen and NetworkClient.connect().	Works locally and on same Wi-Fi/LAN when firewall allows the port.
Unique username registration	Successfully implemented	REGISTER, REGISTER_OK, REGISTER_FAIL.	Duplicate names are rejected.
Online players list	Successfully implemented	USER_LIST broadcast includes username, status, and snake color.	Shows IDLE, IN_GAME, and SPECTATING states.
Player challenge flow	Successfully implemented	CHALLENGE, CHALLENGE_RECEIVED, ACCEPT, REJECT.	Only idle players can start a match.
Snake customization	Successfully implemented	Color selection is stored and sent through CUSTOMIZATION_UPDATE.	Movement keys are local client settings.
Authoritative movement	Successfully implemented	Clients send INPUT; server advances match.	Clients render the received state only.
Pies and health rewards	Successfully implemented	Pie collection increases health by PIE_REWARD.	One pie is kept active on the board.
Obstacles and collisions	Successfully implemented	Static obstacles and collision damage in game_engine.py.	Damage applies for wall, obstacle, self, and snake contact rules.
Timer and winner decision	Successfully implemented	120-second timer and GAME_OVER result.	Winner is higher health or surviving player.
Text chat	Successfully implemented	CHAT_SEND and CHAT_MESSAGE.	Direct messages between users.
Spectator mode	Successfully implemented	SPECTATE_REQUEST and spectator state broadcast.	Spectators receive the same GAME_STATE stream.
Fans / cheering	Successfully implemented	CHEER inserts timed cheer message into GAME_STATE.	Shown beside the chosen player.
Creative crown power	Successfully implemented	Purple power pie grants crown; crown owner damages opponent on contact.	Power pie appears after normal pies and only one crown owner exists.

5. Implementation of Main Functionalities

5.1 Registration, Lobby, and Customization

After connecting, the client sends REGISTER with the chosen username. The server rejects empty or duplicate usernames and closes failed registration attempts. Successful users are stored in the clients dictionary, given IDLE status, and sent to all clients through USER_LIST. Customization is handled by sending the selected snake color to the server. This color is included in lobby updates and applied to the player snake when a match starts.

5.2 Match Creation and Server Game Loop

A player challenges an idle target from the lobby. The target receives a challenge notification and can accept or reject it. On acceptance, the server marks both players IN_GAME, creates the match object, sends GAME_START to both clients, then starts the match loop after a short countdown. The match loop advances at the configured tick rate and sends GAME_STATE updates to both players and all spectators.

5.3 Game Engine Rules

The game board is a 12 x 14 grid. Each snake starts with 50 HP and length 3. Movement is directional; reversing directly into the opposite direction is rejected. The engine computes the next head position, detects collisions with walls, obstacles, self, and the other snake, then applies 5 HP collision damage when required. Pie collection grants 10 HP. If a player reaches 0 HP, the match ends immediately. If time expires first, the higher-health player wins; equal health produces a draw.

5.4 Advanced Features

Text chat is implemented through the server as a direct routing service: a client sends CHAT_SEND with the target and message, and the server sends CHAT_MESSAGE to both the recipient and sender so both histories remain consistent. Spectator mode allows an idle third client to request viewing the active game; the server marks that user SPECTATING and includes the spectator socket in the GAME_STATE broadcast list. Fans are implemented through CHEER messages, which add a temporary cheer message to the shared game-state payload.

The creative feature is the crown power. After five normal pie appearances, an apple appears next and is marked as a power apple and rendered purple. The player who eats it becomes the crown owner. While the crown is active, contact between the crowned snake and the other snake causes collision damage to the non-crowned player. The match state stores only one crown_owner, so both players cannot hold the power at the same time. The crown is lost after the next 3 pies appearances.

6. Testing and Validation

The project was validated by running multiple client windows against one server and testing the full user flow: connection, username registration, duplicate-name rejection, lobby refresh, customization, challenge acceptance/rejection, real-time movement, pie collection, obstacle/wall/snake collisions, chat, spectating, cheering, crown power, and game-over results. The most important validation point is that all clients render the same server-generated GAME_STATE, including snake colors, health values, timer, obstacles, pies, crown ownership, and cheers.

Appendix A - Application Snapshots

This appendix documents the main implemented features through screenshots of the application screens and gameplay states.



Figure A.1. Connection screen: the user enters the server IP address and port before opening a TCP connection.



Figure A.2. Username validation: duplicate usernames are rejected with an error message.

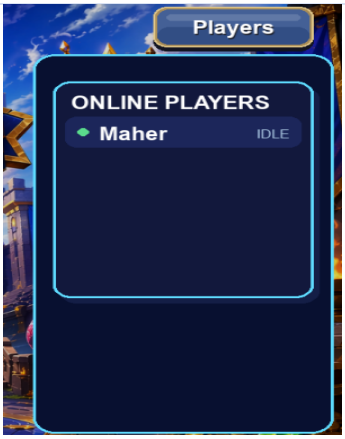


Figure A.3. Lobby: online players are displayed with status and selected snake color.



Figure A.4. Customization: the user selects snake color and custom movement keys.



Figure A.5. Challenge flow: incoming challenge notification with Accept and Reject actions.



Figure A.6. Gameplay: two snakes, health bars, timer, pie, and obstacles are shown.

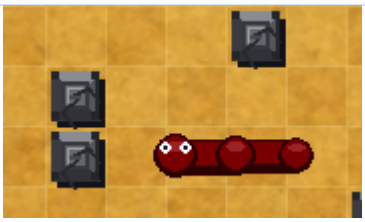


Figure A.7. Collision feedback: damaged snake state after collision damage.



Figure A.8. Text chat: direct messages are exchanged between online users.

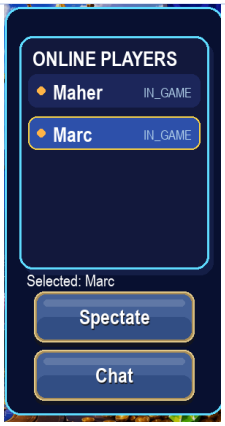


Figure A.9. Spectator entry: idle users can spectate players who are currently in a match.



Figure A.10. Spectator view: a third user watches the ongoing match and can leave the game view.



Figure A.11. Fans feature: spectator cheering displays a temporary support message during gameplay.



Figure A.12. Creative feature: purple power apple appears after normal pie appearances.



Figure A.13. Creative feature: crown power is displayed on the snake that collected the purple pie.



Figure A.14. Result screen: winner, remaining time, and final health scores are displayed.

Appendix B - Project Task Breakdown

The following table summarizes the implementation tasks and the team member mainly responsible for each task.

Task	Mainly Responsible	Deliverable / File Area	Status
Protocol message design, constants, framing, and send/receive helpers	Maher Karam	protocol.py	Completed
TCP server setup, command-line port, socket options, and client threads	Maher Karam	server.py	Completed
Username validation, online-user tracking, and lobby broadcasts	Maher Karam	server.py / USER_LIST	Completed
Challenge flow and match session creation	Maher Karam	server.py matchmaking handlers	Completed
Authoritative game loop and state broadcasting	Maher Karam	server.py run_game_loop	Completed
Board, snake, pie, obstacle, health, timer, and winner models	Marc-Anthony Bou Mikhael	models.py / game_engine.py	Completed
Snake movement rules and invalid reverse-direction prevention	Marc-Anthony Bou Mikhael	game_engine.py	Completed
Collision detection and damage rules	Marc-Anthony Bou Mikhael	game_engine.py	Completed
Pie spawning, health reward, purple power pie, and crown mechanics	Marc-Anthony Bou Mikhael	game_engine.py	Completed
Connection screen and server IP/port input	Alexandre Yammine	connect_screen.py	Completed
Username screen, lobby interface, player list, and challenge buttons	Alexandre Yammine	username_screen.py / lobby_screen.py	Completed
Snake color customization and movement key customization	Alexandre Yammine	customize_screen.py / client_state.py	Completed
Gameplay rendering, health bars, timer, board, snakes, obstacles, pies, crown, and cheers	Alexandre Yammine	game_screen.py / art.py	Completed
Result screen and return-to-lobby flow	Alexandre Yammine	result_screen.py	Completed
Client networking, receive thread integration, and UI state updates	Maher Karam and Alexandre Yammine	network.py / client.py	Completed
Text chat interface and server routing	Alexandre Yammine and Maher Karam	lobby_screen.py / server.py	Completed
Spectator mode and fan cheering	Marc-Anthony Bou Mikhael	server.py / lobby_screen.py / game_screen.py	Completed
Integration testing on multiple clients and bug fixing	All team members	Full application	Completed
Report screenshots and final packaging	All team members	Report appendices / source code zip	Completed